

Benchmark: .NET virtual actor frameworks

21.06.2022

In previous articles we introduced the concept of virtual actors and their potential use in IoT, data stream processing, gaming and other fields. We also compared 4 most popular actor frameworks for the .NET ecosystem: Orleans, Proto.actor, Akka.Net and Dapr. Let's see how they perform in an artificial messaging benchmark.

Note: We were involved to some extent in the development and documentation of the Proto.Actor framework. However, in this article we try to be as objective as possible.

the Orleans part of the benchmark again and were able to get improved performance numbers. The charts in the article are now updated. We also added a note on benchmark results reproducibility.

Goal

The goal of this benchmark is to compare performance of the virtual actor frameworks. We will look at two main aspects:

- Messaging throughput and latency – this includes the overhead the frameworks introduce for message serialization and routing in a cluster.
- Time and memory consumed to activate actors.

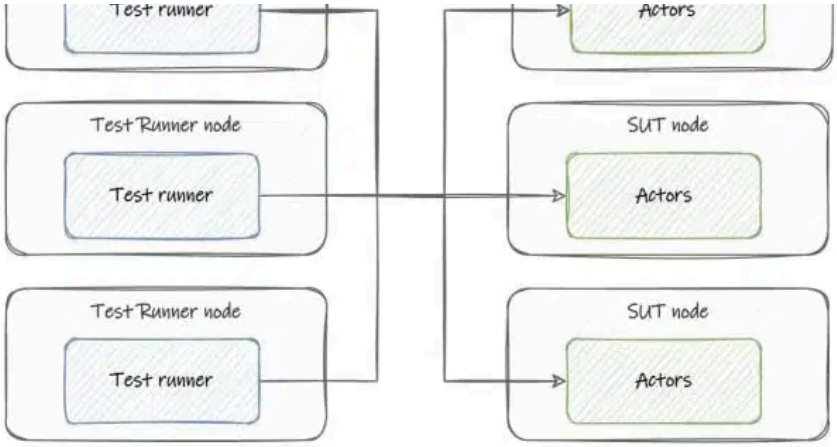
What is also important, we would like to run the benchmark in an environment that resembles a typical production setup - Kubernetes cluster in a public cloud.

Note: When browsing the results, please remember that we're only looking at the framework performance aspect. In a real-world application, you could expect that other factors like: complexity of your business logic, actor persistence, external API calls, etc. will have the most significant impact on the overall performance and memory consumption.

Testing environment

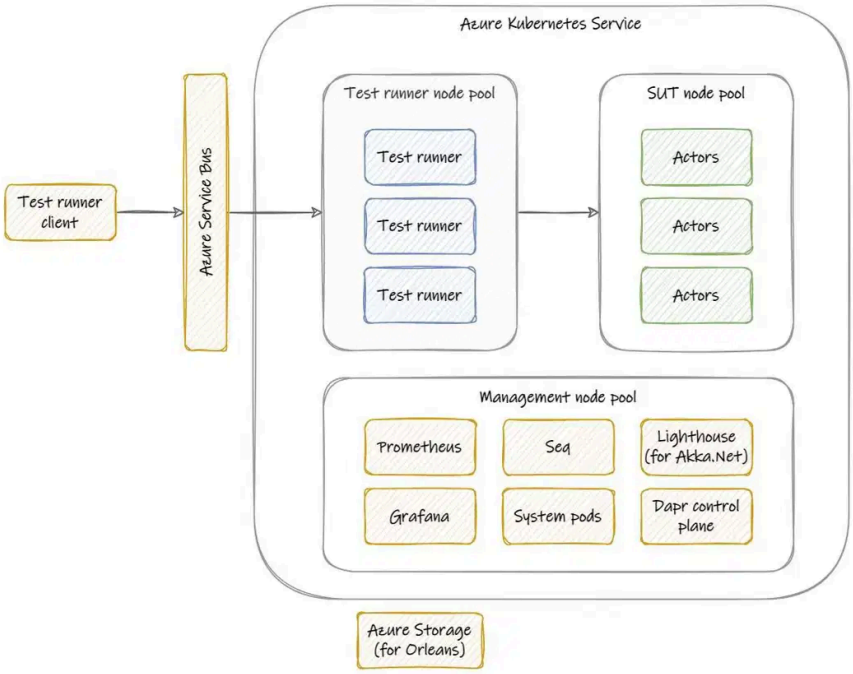
The tests are performed in an environment with 3 machines running the actors (SUT - System Under Test) and another 3 running the clients that send requests to the actors (test runners). The test runners also run code responsible for measuring performance metrics.

The VMs are 4 core / 16GB each, with 10Gbps+ expected network bandwidth (D4s v4 and D4s v5 Azure VMs).



Two additional machines host the supporting components for collecting metrics and logs. The whole test environment is hosted inside of a Kubernetes cluster in Azure (AKS).

The tests are triggered from the developer's machine with a simple tool that coordinates test execution. The commands to start the test are sent over Azure Service Bus.



Messaging test

The first test uses a simple ping - pong message exchange between the test runner and the actor. The test runner

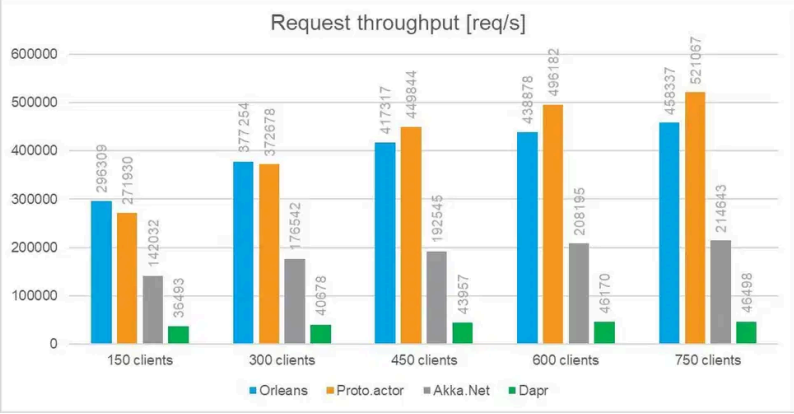
On the client side, we spawn a number of threads (“clients”). Each client communicates with one corresponding actor on the SUT side (there are as many actors in the system, as there are client threads). The client runs ping - pong exchange in a loop. The actors are activated before the benchmark begins to exclude the activation time from the results. The whole test lasts for 5 minutes.

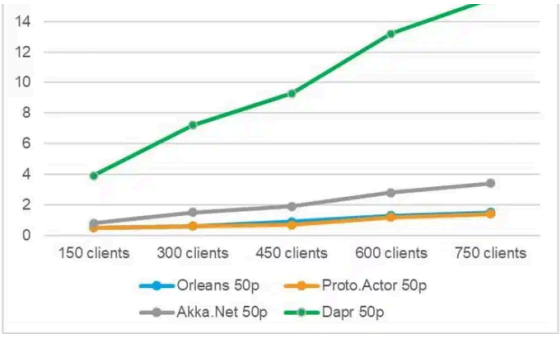
Please note how we’re testing the request - response pattern here. We want to ensure that the message has reached the target actor and that it was able to return a correct response.

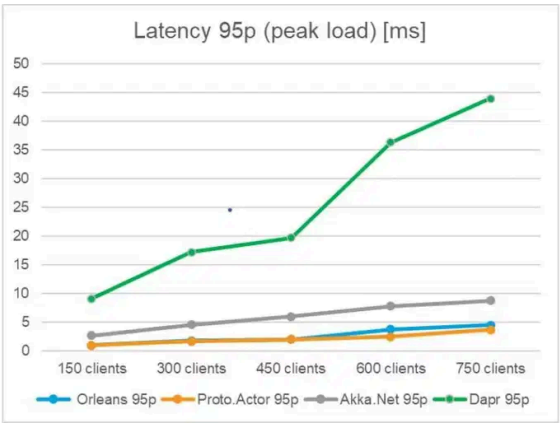
Also note that higher throughput could be achieved with mechanisms like batching or, in the case of some of the frameworks, a more asynchronous exchange of messages. The benchmark assumes a basic implementation only.

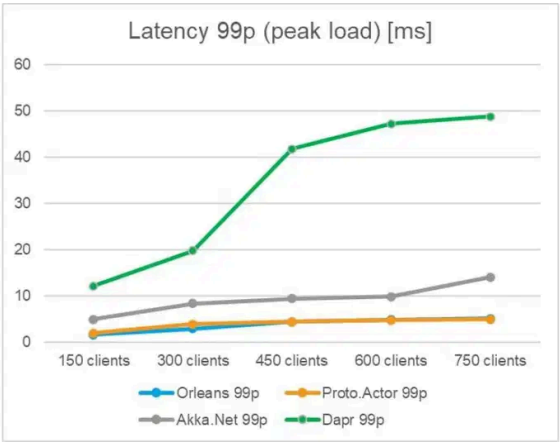
In this test, we measure throughput and latency of the messages. We also look at CPU saturation to detect any potential issues with the test itself.

The diagrams below present results for different number of threads (“clients”) counted as a total across all 3 test runner nodes. The throughput is the sum from all the test runner instances, while latency is an average.









The performance numbers look satisfyingly, with high throughput and low latency for the top 3 of the frameworks. Proto.Actor seems to be a leader in this test while Dapr is definitely an outlier.

CPU usage looks a little awkward at first:



To comment on this pattern:

- With each framework, either SUT or test runner nearly saturates assigned CPU cores. Around 10.5 cores out of 12 are used. This indicates that there are no bottlenecks in the network infrastructure. Note that some CPU on each node is reserved by the kubelet and other system services.
- High CPU usage on test runners implies that the client side of the communication requires processing power comparable with the server side. Also, test runners use some of CPU to collect benchmark metrics.
- There is not that much difference between test runs with different number of client threads. At the same time throughput is increasing with the number of client threads. This is, however, at the expense of increased latency.
- Akka.Net has the highest disproportion between client side and server-side CPU usage. Perhaps this is due to execution of the routing logic on the client side.

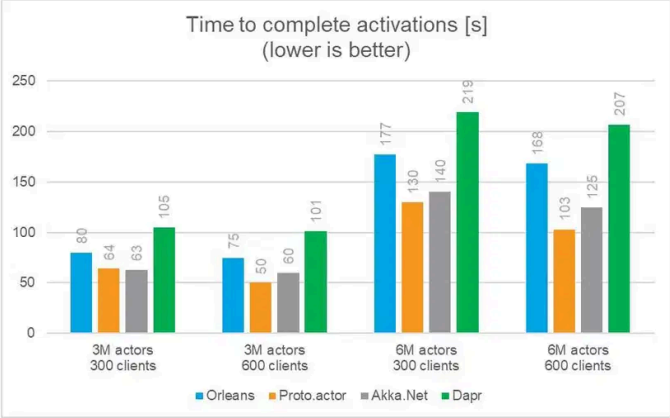
Activation test

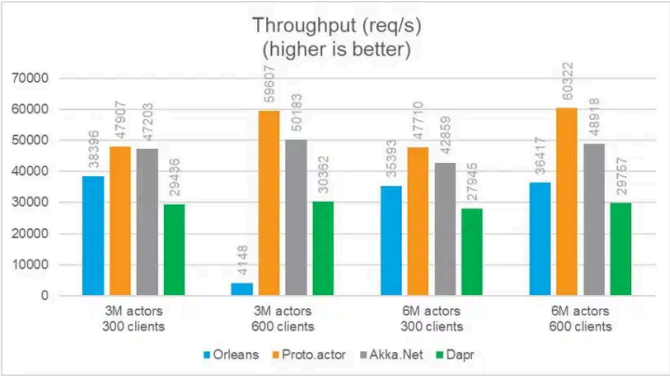
In this test we activate large numbers of actors by sending an empty Ping message to them.

Before the test starts, we generate a number of unique actor ids to be activated. Then we spawn a number of threads (“clients”), that take the ids from the list and activate a corresponding actor. The benchmark completes as soon as the list of ids is empty.

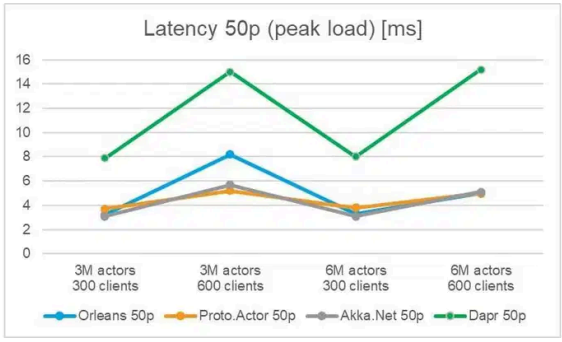
information about the memory used.

The diagrams below present results for different number of threads ("clients") counted as a total across all 3 test runner nodes. The time to complete the activations is max among all the test runner instances. The throughput is the sum, while latency is an average. Total number of activations is a sum across all test runner instances.

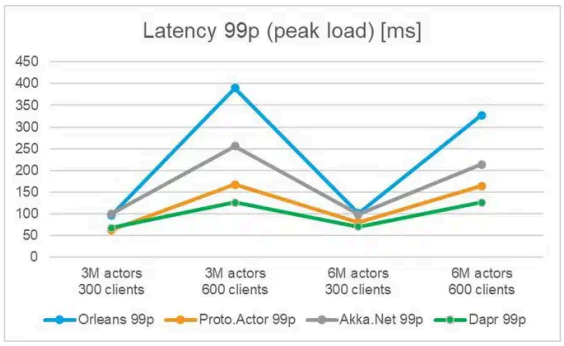


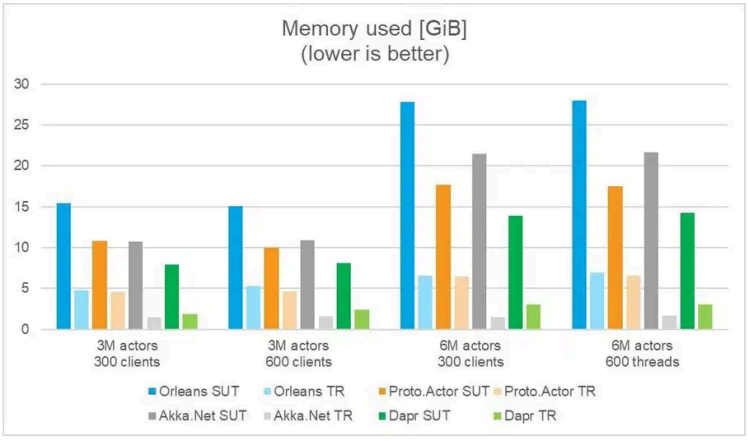


For latency, the 99p varied a lot during the benchmark. Here we present the average of the min and max observed.









The results show that Dapr and Orleans are the slowest when it comes to actor activations, while the other two - Akka.Net and Proto.Actor - have similar results. However, given the huge number of actors, the time to activate them is probably acceptable in all cases.

Let’s also take a look at the memory consumption. The memory is measured as peak total memory used by all the pods belonging to a group.

Here we can see some significant differences with Orleans being the least and Dapr being the most memory efficient.

Conclusion

performance of Dapr is a surprise). Let's repeat it again: in a real world application, the business logic execution, actor persistence, external APIs will dominate the execution time. It is probable, that it's not the actor framework that will be the bottleneck in your solution.

How to chose the right framework? If your use case involves a high throughput message passing or high number of actors being activated in a short time, then the benchmark results may be useful to make the selection. Otherwise, perhaps factors other than performance will play the main role in decision making process. Things like developer friendliness, breadth of use cases covered, ease of deployment, reliability and support are examples of what should be considered.

A note on benchmark results reproducibility

We noticed that after recreating the AKS environment, the benchmark results may differ from previous runs, although the same types of VMs are used. We suspect this is due to the nature of the public cloud, where the test environment is not fully isolated and might be affected by the "noisy neighbor" effect or simply placed on different infrastructure.

Extra: Actor framework configuration details

All benchmarks run on .NET 6 with full PGO enabled, which has proven to improve results for each framework.

Orleans

Version used: 3.6.0. The Orleans deployment is set up as 6 node cluster with Azure Storage clustering provider. The 3 test runners are silos that do not have any grain types registered. All the ping-pong grains are spawned on the 3 SUT silos. The remaining configuration uses default settings.

Proto.Actor

as test runners. The test runners have no actor kinds configured, so they do not host actors themselves.

The benchmark does not use code generated virtual actors, because although they are convenient, they also introduce an overhead of about 5-8%. Additionally, compression is enabled for gRPC connections.

Kubernetes clustering provider is used along with Partition Identity Lookup, which is proto.actor's default identity ownership strategy. The message types are generated from proto files, so protobuf serialization is used. The remaining configuration uses default settings.*Note: Using the alternative Partition Activator Lookup will improve activation throughput ~10-30%, however it has some interesting consequences for cluster balancing. Be sure to read the next blog post in the series to learn more.*

Akka.Net

Version used: 1.4.36. Akka.Net deployment is a 6 node cluster, 3 of them hosting actors, and remaining 3 serving as test runners. Only SUT nodes host actors thanks to proper node role configuration.

The Akka.Net clustering implementation uses seed nodes. We're using the Lighthouse service deployed to management node pool in Kubernetes to ensure the seed nodes are always available and that they do not affect benchmark results.

By default, Akka.Net uses Newtonsoft.Json for serialization, however this has significant impact on the performance. So in the benchmark we're using the Hyperion serializer. Note however, that it is still marked as beta.

The remaining configuration uses default settings.

Dapr

containers to determine which nodes support what actor types. In the test environment, we used 3 service instances hosting the PingPong actor and 3 more acting as clients - all assigned to appropriate node pools.

Features such as tracing, metrics collection and mTLS have been disabled to create similar conditions to other frameworks. Redis was used as the actor state storage, however no state was stored during the test. All other aspects of the configuration were left at their default values.

Marcin Budny, R&D Lead

Marcin is a software architect and developer focused on distributed systems, cloud, APIs and event sourcing. He worked on projects in the field of IoT, telemetry, remote monitoring and management, system integration and computer vision with deep learning. Passionate about latest tech and good music

Let's get in touch!

Etteplan has offices in Finland, Sweden, the Netherlands, Germany, Poland, Denmark and China. Etteplan has also a sales office in the USA. We are a technology service company that, with our thinkers and doers, enable breakthroughs that help industrial businesses change.

Contact us →

Headquarters
Etteplan Oyj
Tekniikantie 4 D
02150 Espoo
Finland

Call center
+358 10 3070

General questions
info@etteplan.com

15100 Lahti
Finland

Services

- → Engineering
- → Etteplan rAIse – Pragmatic industrial AI for smarter businesses
- → Green Transition
- → Nearshoring and Offshoring services
- → Project Management Services
- → Software and Embedded Services
- → Technical Communication
- → Testing and Test Laboratory

Solutions

- → Aftermarket Solutions
- → Asset Development
- → Asset Operations and Maintenance
- → Order to Delivery
- → Production Development
- → Research & Development

Local sites

- → 中国
- → Danmark
- → Suomi
- → Deutschland
- → the Netherlands
- → Polska
- → Sverige
- → Global English

→ Facebook → Instagram → LinkedIn → YouTube

Privacy statement Privacy inquiry Accessibility Invoicing addresses

© Etteplan 2026